SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Theory of Computation **COURSE CODE:** CSA13
COURSE OUTCOME COVERED
CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision
problems (BL:3)
Simulation
Name : **Yuvann Nithish R**
Register No : **192411267**

SIMATS ENGINEERING
ASSESSMENT TOOL

Solutions

Question 1: NFA for Automatic Door System

Formal Definition (5- tuple): N = (Q, Σ, δ, q0, F)

- Q = {Closed, Opening, Open, Closing}
- Σ = {P, N} (P = Person detected, N = No person detected)
- q0 = Closed (start state)
- F = {Open} (accepting state — door successfully open)

Transition Table

State	P	N
→ Closed	{Opening}	{Closed}
Opening	{Opening, Open}	{Open}
*Open	{Open}	{Open, Closing}
Closing	{Opening}	{Closing, Closed}

(→ = start state, * = final/accepting state)

Explanation of Non - determinism:

- From Opening on input P, the system may non -d eterministically stay in Opening (still moving) or advance to Open — modeling the uncertain duration of the door's opening motion.
- From Open on input N, the automaton may either remain in Open (grace period / sensor delay) or transition to Closing — this is the key non -d eterministic choice described in the problem.
- From Closing on input N, the system may stay Closing or complete the cycle back to Closed.

State Diagram (textual):
Closed — P⟶ Opening —P⟶ Open —N⟶ Closing — N⟶ Closed, with self-l oops: Closed on N, Opening on P
(self- loop, non- det.), Open on P (self-l oop) and on N (self-l oop, non -det.), Closing on N (self-l oop, non -d et.).

Question 2: Finite Automaton for Traffic Light Control System
Formal Definition: M = (Q, Σ, δ, q0, F)
● Q = {Red, Green, Yellow}
● Σ = {T} (T = timer signal / timeout event that triggers the next transition)
● q0 = Red (start state)
● F = Q (every state is a valid/accepting operating state — this is a cyclic controller, not a language
acceptor)

Transition Table

State	Input T (timer expires)	Duration
→ Red	Green	30 sec
Green	Yellow	25 sec
Yellow	Red	5 sec

Cycle:
Red —T⟶ Green — T⟶ Yellow — T⟶ Red — T⟶ ... (repeats indefinitely; the automaton has no final halting state
since traffic control runs continuously).
Handling continuous cycling:
Each state has an internal timer. Once the timer for a state expires, input symbol T is generated internally, forcing
a deterministic transition to the next state in the fixed sequence. Because δ is a total function returning exactly one
next state for T, the automaton is deterministic and cycles forever without ever reaching a dead or terminal
condition.
Extension – Pedestrian Crossing:

Add a new state Pedestrian_Walk , reachable from Red on a pedestrian -b utton input signal B: δ(Red, B) =
Pedestrian_Walk. This state runs its own timer (walk time) and then returns to Red before the normal cycle
resumes: δ(Pedestrian_Walk, T) = Red.

Extension – Emergency Vehicle Override:

Add an input symbol E (emergency signal) that can interrupt any state: δ(Green, E) = Red and δ(Yellow, E) =
Red. This forces the intersection to Red immediately regardless of the current timer, after which a dedicated
Emergency state can hold Red for all conflicting directions until E is withdrawn, then normal cycling resumes
from Red.

Question 3: DFA Accepting Strings Ending with "ab" over {a, b}

Formal Definition: M = (Q, Σ, δ, q0, F)

- Q = {q0, q1, q2}
- Σ = {a, b}
- q0 = start state (no relevant suffix seen yet)
- F = {q2} (last two symbols read were "ab")

Transition Table

State	a	b
→ q0	q1	q0
q1	q1	q2
*q2	q1	q0

State meaning:

- q0 — initial state / last symbol was not 'a'
- q1 — last symbol read was 'a' (potential start of "ab")

● q2 — accepting state; the last two symbols read were exactly “ab”

Trace for W = aaabab:

Step	Input	Current State	Next State
1	a	q0	q1
2	a	q1	q1
3	a	q1	q1
4	b	q1	q2
5	a	q2	q1
6	b	q1	q2

Result: Final state reached = q2 ∈ F → the string “aaabab” is ACCEPTED (it ends with “ab”).

Question 4: DFA Accepting Exactly {“c”, “bc”, “bcaaa”} over {a, b, c}
Formal Definition: M = (Q, Σ, δ, q0, F)
● Q = {q0, q1, q2, q3, q4, q5, q6, qd}
● Σ = {a, b, c}
● q0 = start state
● F = {q1, q3, q6} (accept exactly “c”, “bc”, “bcaaa” respectively)
● qd = dead/trap state for any string that deviates from the three valid patterns

Transition Table

State	a	b	c	Meaning
→ q0	qd	q2	q1	start
*q1	qd	qd	qd	accepts “c”
q2	qd	qd	q3	read ‘b’
*q3	q4	qd	qd	accepts “bc”
q4	q5	qd	qd	read “bca”
q5	q6	qd	qd	read “bcaa”
*q6	qd	qd	qd	accepts “bcaaa”
qd	qd	qd	qd	dead state (trap)

Explanation:
● q0 — c→ q1 accepts the exact string “c”; any symbol after reaching q1 sends the automaton to qd since a
longer string is no longer equal to “c”.
● q0 — b→ q2 —c→ q3 accepts the exact string “bc”.
● q3 — a→ q4 — a→ q5 — a→ q6 accepts the exact string “bcaaa” (exactly three a’s after “bc”, no more, no
less).

● Every other symbol combination (wrong branch, extra symbols, or premature stop that isn't q1/q3/q6)
leads to qd , which loops on all inputs and is never accepting — guaranteeing only the three target strings
are accepted.

Question 5: DFA Accepting All Strings Starting with 'a' or 'b' over {a, b}

Formal Definition: M = (Q, Σ, δ, q0, F)

● Q = {q0, q1}
● Σ = {a, b}
● q0 = start state (nothing read yet)
● F = {q1} (at least one symbol has been read)

Transition Table

State	a	b
→ q0	q1	q1
*q1	q1	q1

Explanation:

Since Σ = {a, b}, every non-e mpty string necessarily begins with either 'a' or 'b'. As soon as the first symbol is
read, the DFA moves from q0 to the accepting state q1 and remains there (self-l oop on both a and b) for the rest
of the string, regardless of what follows. The empty string leaves the automaton in q0, which is non -a ccepting, so
it is correctly rejected.

Note on the dead state: A separate trap state is not strictly required here because both symbols from both states

lead to a valid accepting state; if the simulator (e.g., Autosim) requires δ to be fully defined and a designated fail
case for malformed/non -a lphabet input, an addit ional qd state can be added such that δ(q0, x) = qd and δ(q1, x) =
qd for any x ∉ {a, b}.